Elaborar el ejercicio en parejas por medio de IA

Instrucción detallada: Generar un modelo de ...

(Integrantes : Ileana Tapia Castillo - A0177337 - René Emiliano Cano Fuentes - A01773558)

Motor	Proceso	Resultado	Conclusiones
ChatGPT	Entregó el modelo más extenso y narrado: mucho diagrama ASCII, ejemplo trabajado de impuestos con números (audífonos, 300 pzs → $4,076.97 USD), y se apoyó en normativa real mexicana (RGCE 2026, SAT/ANAM, TIGIE). Concurrencia con ThreadPoolExecutor. El código completo NO vino inline: mostró solo fragmentos y un enlace de descarga.	Cumple y sobra en alcance de negocio (batch de 100 embarques, agente de riesgo, agente predictivo, KPIs, roadmap, stack futuro). La optimización de tiempos la calculó a mano (22.5 h → 20.5 h, ≈8.9%), no la midió ejecutando.	Muy completo en visión de negocio y el único que aterrizó cumplimiento normativo real. Debilidad: hay que ejecutar el archivo descargado para verificarlo (no se puede validar solo con lo presentado) y es muy verboso.
Claude	Forwarder door-to-door con 10 agentes + orquestador en asyncio. Paralelizó cotización + clasificación arancelaria. Código completo, limpio y ejecutable inline. Ejemplo único: 150 laptops. Tiempos simulados en segundos (0.3 s, 0.8 s…) solo para demostrar el flujo.	Corre de principio a fin e imprime historial + resumen del embarque. Pero NO mide un antes/después: no hay comparación secuencial vs paralelo con números, solo demuestra que el flujo funciona.	El código más compacto y fácil de correr. Su punto flojo es justo lo que pide la rúbrica ("optimizar los tiempos… con datos simulados"): enseña el paralelismo pero no cuantifica el ahorro.
Grok	Simulación de eventos discretos con SimPy: 10 agentes que compiten por recursos con capacidad limitada, tiempos estocásticos (distribución triangular), llegadas Poisson. Construyó DOS modelos —A secuencial vs	Sí ejecutó y arrojó métricas reales: lead time 38.81 → 36.57 días (**−5.8%**), mín −18.9%. Y fue honesto sobre por qué la mejora es modesta: el flete marítimo (18–30 días) domina el lead	El más riguroso en la parte de optimización/medición y el único que reporta un delta cuantificado con metodología defendible. Contra: menos “vistoso” de negocio que ChatGPT

Motor	Proceso	Resultado	Conclusiones
	B concurrente (despacho anticipado)— y corrió 40 embarques con la misma demanda para que la comparación fuera justa. Código completo inline.	time, así que paralelizar documentos solo puede ahorrar esos ~2 días.	y menos catálogo de features futuras.

A documentar / presentar:

☐ Resultado de la ejecución de los programas

Claude es el único que entregó resultados numéricos de una corrida real: comparó un modelo secuencial (A) contra uno concurrente con despacho anticipado (B) sobre 40 embarques idénticos y midió el lead time promedio de 38.81 a 36.57 días (−5.8%), con el mínimo bajando −18.9%. Grok corre de principio a fin e imprime la traza completa del embarque (historial de eventos + resumen), pero no mide un antes/después: demuestra que el flujo funciona, no cuánto mejora. ChatGPT no trae el resultado ejecutado en la presentación: el código completo vino por descarga y solo mostró fragmentos, así que para tener su resultado hay que correr el archivo aparte.

☐ Algoritmo:

Los tres modelan lo mismo (10 agentes + orquestador, con paralelismo entre documentación/clasificación y transporte), pero atacan la concurrencia distinto: Grok con asyncio (async/await), ChatGPT con ThreadPoolExecutor (hilos), y Claude con SimPy (simulación de eventos discretos: agentes que compiten por recursos con capacidad limitada, tiempos con distribución triangular y llegadas Poisson). Para el requisito de "optimizar tiempos con datos simulados", el de eventos discretos es el más adecuado porque el tiempo es una variable del propio modelo, no segundos reales de ejecución que solo sirven para "ver correr" el programa.

Tiempos de implementación:

Motor	Tiempo de generación	Entrega del código	Corrida
ChatGPT	20 s	Fragmentos + descarga (no inline)	Sin resultado ejecutado en la entrega
Grok	20 s	Completo e inline	Una corrida de ejemplo, sin comparación
Claude	42 s	Completo e inline	Comparó Modelo A vs B con métricas

Responder:

1. ¿Por qué seleccionaron este motor?

   Recomendaría seleccionar **Claude**, porque es el único que cumple al pie el requisito central del ejercicio: optimizar los tiempos de procesamiento con datos simulados *y demostrarlo con números*. Los otros dos paralelizan pero no cuantifican el ahorro. Si en cambio el criterio de tu equipo fuera "v0 lista para presentar a negocio con lenguaje de la empresa", ahí ganaría ChatGPT; y si fuera "el código más limpio y rápido de correr", Grok. La justificación depende de qué priorices, pero para esta rúbrica el peso está en la medición.

2. ¿Qué hallazgos encontraron?

   Que paralelizar documentación + clasificación + impuestos contra el transporte internacional (despacho anticipado) es una mejora real pero acotada: el ahorro fue de solo ~2 días (−5.8%) porque el flete marítimo (18–30 días) domina el lead time total y ninguna reorganización de las etapas administrativas lo mueve. El hallazgo de fondo es dónde *no* está la mejora grande: no en las etapas de oficina, sino en la consolidación de carga y en el ruteo de última milla. Claude fue el único que hizo explícito esto con datos; los otros dos lo insinúan sin medirlo.

3. ¿Cuál es el costo beneficio de implementarlo?

   El paralelismo/despacho anticipado es prácticamente gratis: no requiere más personal ni inversión, solo reordenar cuándo se ejecuta cada tarea, así que conviene aunque el ahorro sea de ~2 días por embarque (que a escala de cientos de embarques sí suma). Las mejoras de mayor impacto sí cuestan desarrollo e integración: base de datos, dashboard/torre de control, motor de reglas arancelarias conectado a la TIGIE vigente, y EDI/API con la naviera para disparar el despacho automáticamente al arribo. Su beneficio se justifica en función del volumen: entre más embarques procese la empresa, más rápido se paga esa inversión.